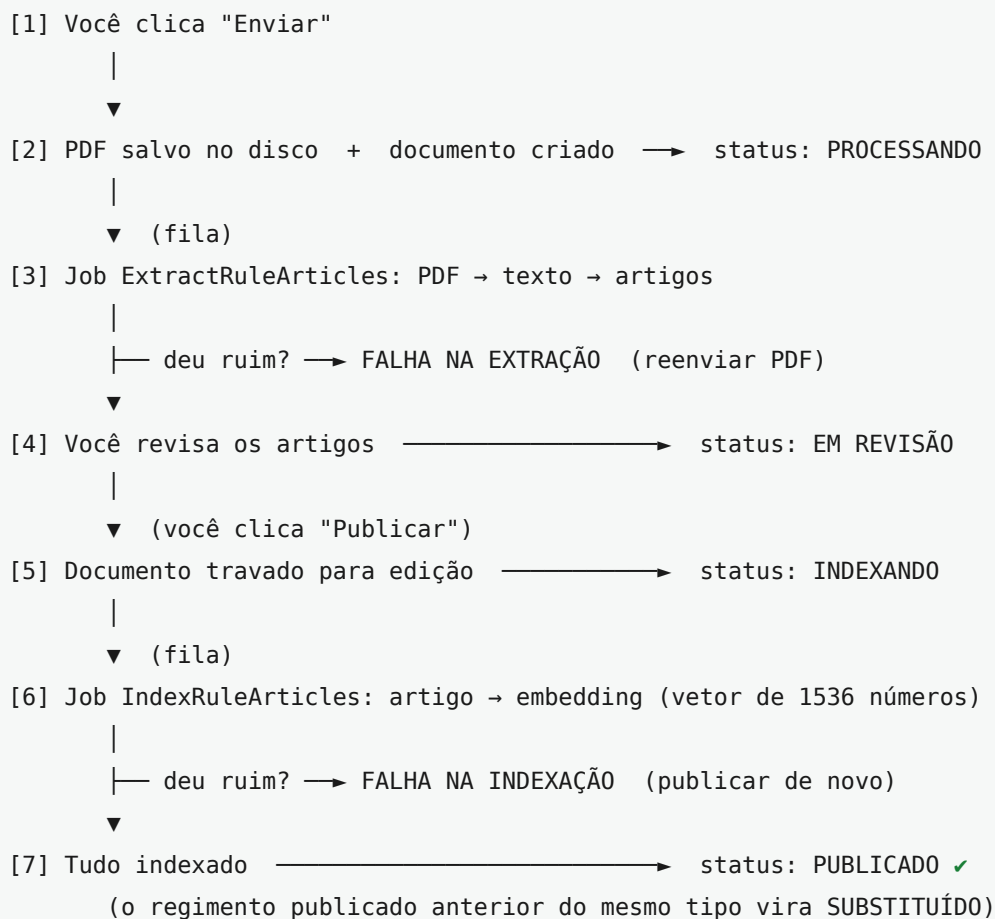


Do PDF aos embeddings: o caminho do upload

Resumo: você envia o PDF → o PDF é salvo → um job na fila extrai o texto e corta em artigos → você revisa → clica em Publicar → outro job transforma cada artigo em um vetor (embedding) → o documento fica **Publicado** e o agente já consegue pesquisar nele.

O mapa (guarde só isto)



Duas coisas para lembrar:

1. Tem **2 jobs de fila**. Sem worker rodando, nada anda.
2. Tem **1 parada humana** (a revisão). O sistema **não** gera embeddings sozinho depois da extração: alguém precisa clicar em Publicar.

Antes de começar: ligue o worker

Sem isso, o documento fica em "Processando" para sempre.

```
./vendor/bin/sail artisan queue:work
```

A fila usa o banco (`QUEUE_CONNECTION=database`). Os jobs ficam na tabela `jobs` até o worker pegá-los.

Parada 1 — O clique em "Enviar"

O que acontece: o navegador manda o PDF para o Livewire, que guarda o arquivo numa pasta temporária. Quando você clica em Enviar, o método `uploadDocument` é chamado.

Onde está: `resources/views/pages/rule-documents.blade.php` → `uploadDocument()`

Validações:

- Tipo: `regimento` ou `convencao`
- Título: obrigatório, até 255 caracteres
- Arquivo: obrigatório, só PDF

⚠ **Pegadinha real deste projeto:** o método se chamava `upload`. Esse nome já é usado pelo JavaScript do Livewire (`$wire.upload(...)`), então o clique chamava a função errada e quebrava em silêncio. **Nunca chame uma ação Livewire de `upload`.**

Parada 2 — Salvar o PDF e criar o documento

O que acontece:

1. O PDF é gravado em `storage/app/private/rule-documents/{id_do_condominio}/`.
2. Uma linha é criada em `rule_documents` com status **processando**.
3. O job `ExtractRuleArticles` é colocado na fila.

Onde está: `app/Services/RuleDocuments/RuleDocumentService.php` → `upload()`

Detalhe bom: se falhar ao salvar no banco, o arquivo é apagado do disco. Não sobra lixo.

Como saber que funcionou: aparece o toast *"PDF enviado. Os artigos estão sendo extraídos."*

Parada 3 — Extrair o texto e cortar em artigos (job)

Onde está: `app/Jobs/ExtractRuleArticles.php`

Passo a passo:

#	O que faz	Ferramenta
1	Lê o PDF do disco	Storage
2	Tira o texto do PDF	smalot/pdfparser
3	Corta o texto em artigos	RuleArticleSplitter
4	Apaga artigos antigos e salva os novos	transação no banco
5	Muda o status para em_revisao	—

Como o texto vira artigos (RuleArticleSplitter)

A regra é simples: **cada linha que começa com um cabeçalho abre um artigo novo.**

Começa com...	Vira a referência
Art. 12 ou Art. 12º	Art. 12 / Art. 12º
Artigo 3	Art. 3
Cláusula 9ª ou Cl. 9a	Cl. 9ª

- Tudo que vem depois do cabeçalho, até o próximo cabeçalho, é o **corpo** do artigo.
- Parágrafos (§ 1º , Parágrafo único) ficam **dentro** do artigo, em linha própria.
- Texto antes do primeiro artigo com 200 caracteres ou mais vira o artigo **"Preâmbulo"**.
- Nenhum cabeçalho encontrado? O PDF inteiro vira um artigo só, chamado **"Documento"**.
- Art. 5 - Horário de silêncio → o trecho depois do - vira **título** (se tiver até 80 caracteres).

Por que cortar por artigo? Cada artigo vai virar um embedding. Artigo é uma unidade de sentido e já vem com a "citação" pronta ("conforme o Art. 34..."). Isso é *chunking* guiado pela estrutura do documento.

Se der errado → status `falha_extracao` :

- O parser não consegue ler o PDF (arquivo corrompido, protegido etc.).
- O PDF não tem texto, porque é **escaneado** (é só uma imagem). A mensagem é: *"Não foi possível extrair texto do PDF (pode ser um documento escaneado)."*
- O job estourou o tempo (600 s) ou o worker caiu.

Como resolver: botão **"Reenviar PDF"** → `reupload()` troca o arquivo e coloca o job na fila de novo.

Parada 4 — Revisão humana (status: Em revisão)

O que acontece: o síndico vê os artigos extraídos e pode:

- ✎ editar a referência, o título e o corpo
- + adicionar um artigo
- ✕ apagar um artigo
- ⬆️ mover um artigo para cima ou para baixo

Onde está: `RuleDocumentService` → `updateArticle()`, `addArticle()`, `deleteArticle()` e `moveArticle()`

Regra de ouro: só dá para editar enquanto o status for **em_revisao**. Depois de publicado, os artigos ficam somente leitura.

Por que existe essa parada? O corte automático nunca é perfeito. Consertar agora sai barato; depois que o embedding é gerado, texto errado vira resposta errada do agente.

→ Para observar em aula com `aula/regimento-interno-condominio.pdf` :

- Os títulos "Capítulo X — ..." **não** são cabeçalhos. Eles acabam colados no fim do artigo anterior.
- Os Anexos (tabela de multas, horários, contatos, FAQ) vêm depois do Art. 207, então vão **todos para dentro do Art. 207**. Resultado: um artigo gigante e um embedding "diluído".
- **Artigo fantasma:** no Art. 196, a quebra de linha do PDF deixou `art. 1.337`, parágrafo único... no começo de uma linha. O splitter não diferencia maiúsculas e minúsculas e entende `1.337` como número `1`. Resultado: um **"Art. 1" falso** no meio do documento, com o fim do Art. 196 dentro dele. Resultado real: **209 artigos** (Preâmbulo + 207 + 1 fantasma).
- Bom exercício: apagar o fantasma, separar os anexos em artigos próprios e comparar as respostas do "Testar pergunta" antes e depois.

Parada 5 — Clicar em "Publicar"

Onde está: `RuleDocumentService` → `publish()`

Checagens (dentro de uma transação, com a linha travada):

1. O status é **em_revisao** ou **falha_indexacao**? Se não for → bloqueia.
2. Tem pelo menos 1 artigo? Se não tiver → *"Adicione ao menos um artigo antes de publicar."*

Se passar:

- Status → **indexando**
- Guarda quem publicou (`published_by_user_id`)

- Coloca o job `IndexRuleArticles` na fila

Por que travar a linha (`lockForUpdate`)? Dois cliques rápidos em Publicar não podem gerar dois jobs brigando.

Parada 6 — Gerar os embeddings (job)

Onde está: `app/Jobs/IndexRuleArticles.php`

O que é embedding, em uma frase: uma lista de 1536 números que representa o **significado** de um texto. Textos com sentidos parecidos têm listas parecidas.

Passo a passo:

1. Pega os artigos **sem embedding** (`whereNull('embedding')`), em lotes de **100**.
2. Monta o texto de cada artigo assim:

```
Art. 34
Horário de silêncio      ← título (se houver)
De domingo a quinta...   ← corpo
```

3. Manda o lote para a OpenAI (`text-embedding-3-small`, 1536 dimensões) via `Laravel\Ai\Embeddings`.
4. Confere se voltou **um vetor por artigo**. Se a quantidade não bater → erro.
5. Salva cada vetor na coluna `embedding` (tipo `vector(1536)`, do pgvector) e grava `embedded_at`.

Por que em lotes? São menos chamadas à API, e ela é mais rápida e mais barata.

Por que só os que não têm embedding? Se falhar no meio, na próxima tentativa só os que faltam são processados. Nada é refeito à toa.

Se der errado → status `falha_indexacao`:

- A OpenAI está fora do ar, a chave é inválida ou o limite de uso estourou.
- O número de vetores veio diferente do número de artigos.
- O job estourou o tempo (600 s).

Como resolver: clicar em **Publicar** de novo, porque `publish()` aceita `falha_indexacao`.

Parada 7 — Publicado ✓

Onde está: `RuleDocumentService` → `completePublication()`

Última conferência (com trava no condomínio e no documento):

- O status ainda é **indexando**?
- Todos os artigos têm embedding?

Se estiver tudo certo:

1. O documento publicado anterior **do mesmo tipo**, no mesmo condomínio, vira **substituído**.
2. Este documento vira **publicado** e recebe `published_at`.

Resultado: só existe **1 regimento publicado** por condomínio. A busca só enxerga documentos publicados.

Bônus — E quando alguém faz uma pergunta?

Onde está: `app/Services/RuleDocuments/RuleSearchService.php`

1. A pergunta ("posso ter cachorro?") também vira um embedding, com o **mesmo modelo** e as **mesmas 1536 dimensões**.
2. O Postgres compara esse vetor com os vetores dos artigos publicados.
3. Volta até 5 artigos (máximo de 10) com similaridade $\geq 0,5$ (`RAG_MIN_SIMILARITY`).
4. O agente responde citando o artigo.

Pergunta e artigos **precisam** usar o mesmo modelo de embedding. Se você trocar o modelo, precisa reindexar tudo.

Cola rápida: status

Status	Significa	O que fazer
● Processando	Esperando ou rodando a extração	Conferir se o worker está ligado
● Falha na extração	Não conseguiu ler o PDF	Reenviar um PDF com texto selecionável
● Em revisão	Artigos extraídos	Revisar e Publicar
● Indexando	Gerando os embeddings	Aguardar (worker ligado)
● Falha na indexação	Erro na API de embeddings	Ver o erro e Publicar de novo
● Publicado	Pesquisável pelo agente	—
● Substituído	Um documento mais novo tomou o lugar	—

Cola rápida: travou, onde olho?

Sintoma	Onde olhar
Clico em Enviar e nada acontece	<code>storage/logs/browser.log</code> (erro de JavaScript/Livewire)
Fica em "Processando" para sempre	O worker está rodando? Tem job parado na tabela <code>jobs</code> ?
Falhou e quero ver o motivo	Coluna <code>processing_error</code> em <code>rule_documents</code>
Job morreu	Tabela <code>failed_jobs</code> e <code>storage/logs/laravel.log</code>
Busca não acha nada	O documento está publicado ? A similaridade está abaixo de 0,5?

Arquivos do fluxo (em ordem)

1. `resources/views/pages/rule-documents.blade.php` — tela e ações do Livewire
2. `app/Services/RuleDocuments/RuleDocumentService.php` — regras e mudanças de status
3. `app/Jobs/ExtractRuleArticles.php` — PDF → texto → artigos
4. `app/Services/RuleDocuments/RuleArticleSplitter.php` — onde corta os artigos
5. `app/Jobs/IndexRuleArticles.php` — artigos → embeddings
6. `app/Services/RuleDocuments/RuleSearchService.php` — pergunta → artigos parecidos
7. `config/condo.php` (`rag`) e `config/ai.php` (`embeddings`) — números e modelo